PGJ

Java · Objektorientierte Programmierung · Aufgaben & Lösungen

Aufgabe 1 & 2

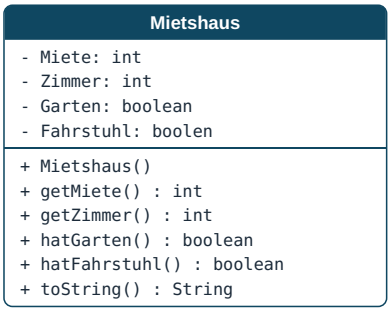
1 Projekt anlegen

Erstellen Sie ein Java-Projekt und benennen Sie es mit Vor- und Nachnamen. Legen Sie im Projekt für die Aufgaben 3 und 4 jeweils ein eigenes Packet an. **Implementieren Sie ausschließlich in diesem Projekt!**

2 Klasse Mietshaus

a)

Implementieren Sie folgende Klasse:



Alle Attribute haben immer zufällige Werte und dürfen nicht vom Benutzer beim Erzeugen angegeben werden!

- **Miete:** 120-1500€
- **Zimmer:** 1-10
- **Fahrstuhl** bzw. **Garten:** true / false

Implementieren Sie die Methode “toString()”, um ein Mietshaus genau wie folgt auszugeben:

```
Miete: 214€, Zimmer:7, Garten: nein, Fahrstuhl: ja
```

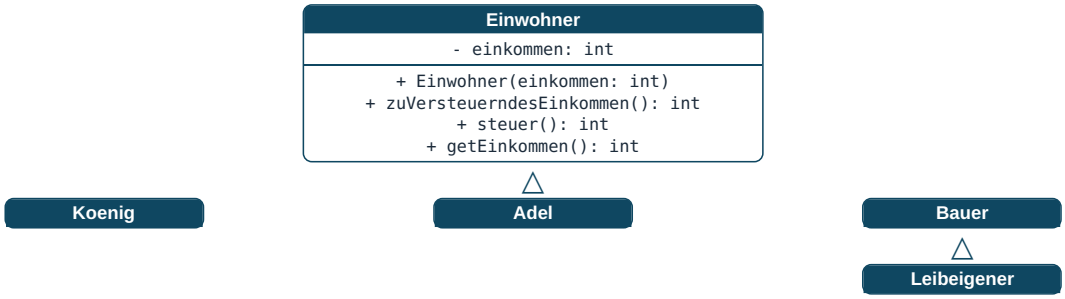
b)

Erzeugen Sie in einer Testklasse 100 Mietshäuser und zerigen Sie alle Häuser auf der Konsole an. Nummerieren Sie alle Häuser von 1-100

Aufgabe 3

3 Kaiserreich: Einwohner & Steuern

In einem mittelalterlichen Kaiserreich soll das Finanz- und Steuerwesen auf EDV umgestellt werden. Die verschiedenen Bevölkerungsgruppen werden durch nebenstehende Klassenhierarchie modelliert:



- **Superklasse:** Einwohner
- Koenig, Adel und Bauer stehen auf derselben Ebene (Vererbung von Einwohner)
- Leibeigener ist der Klasse Bauer untergeordnet

3a Klassenhierarchie implementieren

Einwohner ist eine Oberklasse mit dem int-Attribut **einkommen**, welches das tatsächliche Jahreseinkommen des Einwohners in Talern angibt.

Ergänzen Sie die Klassen, soweit notwendig um die Objektmethoden **zuVersteuerndesEinkommen()** und **steuer()**. Dabei soll so viel wie möglich ererbt werden und nur dort wo notwendig, klassenspezifisch implementiert werden.

`int zuVersteuerndesEinkommen()`	Liefert das zu versteuernde Einkommen
`int steuer()`	Ermittelt und liefert die für jeden Einwohner des Kaiserreichs korrekte Steuer gemäß den folgenden vom Kaiser aufgestellten Vorschriften:

1. Sofern dieses Gesetz nichts gegentiliges aussagt, hat jeder Einwohner sein gesamtes Jahreseinkommen zu versteuern.
2. Jeder Einwohner hat 10% seines zu versteuernden Einkommens als Steuer zu entrichten. Der Steuerbetrag wird auf ganze Taler abgerundet, jedoch beträgt die Steuer immer mindestens 1 Taler.
   Wichtig: Da jährliche Änderungen bei der Steuerberechnung zu erwarten sind, darf diese Vorschrift nur änderungsfreundlich an einer Stelle der Klassehierarchie implementiert werden
3. Der Kaiser zahlt auch für sein steueroflichtiges Einkommen keine Steuern
4. Für Angehörige des Adels beträgt die Steuer mindestens 20 Taler, auch wenn das Einkommen geringer ist.
5. Bei Leibeigenen sind 12 Taler des Jahreseinkommens steuerfrei
6. Für den Bauern gibt es keine vom Einwohner abweichende Regeln.

3b Finanzamt

Implementieren Sie das Finanzamt des Königreichs als Klasse **Finanzamt** mit der Klassenmethode **berechneSteuerEinnahmen()**. Die Methode berechnet die gesamten Steuereinnahmen für alle Einwohner, die in einem Array **Einwohner[]** verwaltet werden.

3c Testen

Testen Sie ihr Programm mit den Testdaten in der Tabelle und geben Sie die gesamten Steuereinnahmen im Königreich auf der Konsole aus.

Einwohner	Einkommen
Kaiser	1.000.000 Taler
Adliger (reich)	30.000 Taler
Adliger (arm)	150 Taler
Bauer	2.000 Taler
Leibeigener 1	150 Taler
Leibeigener 2	5 Taler

Aufgabe 4

4 Abstrakte Klassen & Interfaces

1. Was ist der Hauptunterschied zwischen einer abstrakten Klasse und einem Interface?
2. Können abstrakte Klassen Mehrfachvererbung unterstützen?
3. Wie können abstrakte Methoden erstellt und implementiert werden?
4. Wann ist es sinnvoll, eine abstrakte Klasse anstelle eines Interfaces zu verwenden?
5. Kann eine Klasse gleichzeitig eine abstrakte Klasse erweitern und ein Interface implementieren?
6. Können Variablen in einem Interface deklariert weren?
7. Gibt es Unterschiede in der Sichtbarkeit (Zugriffsmodifizierer) von Methoden und Feldern in abstrakten Klassen und Interfaces?

Lösungen

Alle Aufgaben

Die Original-Mail enthält keine Lösungen. Alle Lösungen sind eine ergänzte Musterlösung.

Aufgabe 1 & 2

1 Lösung: Projekt anlegen (Musterlösung)

Projektstruktur (Projektname = Vor- und Nachname, z. B. *MaxMustermann*):

```
MaxMustermann/
└── src/
    ├── Mietshaus.java          (Aufgabe 2)
    ├── MietshausTest.java      (Aufgabe 2)
    ├── aufgabe3/               (Package Aufgabe 3)
    │   ├── Einwohner.java, Koenig.java, Adel.java,
    │   ├── Bauer.java, Leibeigener.java
    │   └── Finanzamt.java, Main.java
    └── aufgabe4/               (Package Aufgabe 4)
        └── Beispiel.java
```

Jede öffentliche Klasse steht in einer eigenen Datei. In der Ansicht oben sind die Klassen Koenig, Adel, Bauer und Leibeigener zur besseren Übersicht in einem Block gezeigt.

2 Lösung: Klasse Mietshaus (Musterlösung)

a)

```java
import java.util.Random;

public class Mietshaus {
    private static final Random RND = new Random();

    private int miete;
    private int zimmer;
    private boolean garten;
    private boolean fahrstuhl;

    public Mietshaus() {
        miete = 120 + RND.nextInt(1381);   // 120 bis 1500
        zimmer = 1 + RND.nextInt(10);      // 1 bis 10
        garten = RND.nextBoolean();
        fahrstuhl = RND.nextBoolean();
    }

    public int getMiete() { return miete; }
    public int getZimmer() { return zimmer; }
    public boolean hatGarten() { return garten; }
    public boolean hatFahrstuhl() { return fahrstuhl; }

    @Override
    public String toString() {
        return "Miete: " + miete + "€, Zimmer:" + zimmer
             + ", Garten: " + (garten ? "ja" : "nein")
             + ", Fahrstuhl: " + (fahrstuhl ? "ja" : "nein");
    }
}
```

b)

```java
public class MietshausTest {
    public static void main(String[] args) {
        for (int i = 1; i <= 100; i++) {
            Mietshaus haus = new Mietshaus();
            System.out.println(i + ": " + haus);
        }
    }
}
```

Beispielausgabe: `1: Miete: 214€, Zimmer:7, Garten: nein, Fahrstuhl: ja`

Aufgabe 3

3 Lösung: Einwohner & Steuern (Musterlösung)

a)

Die 10-%-Regel (Regel 2) steht **nur in Einwohner.steuer()**. Abweichungen werden über kleine Hook-Methoden bzw. Überschreiben gelöst: Adel ändert nur `mindestSteuer()`, Leibeigener nur `zuVersteuerndesEinkommen()`, Koenig überschreibt `steuer()`.

```
package aufgabe3;

public class Einwohner {
    private static final int STEUERSATZ_PROZENT = 10;   // einzige Stelle fuer die Regel

    private final int einkommen;

    public Einwohner(int einkommen) {
        this.einkommen = einkommen;
    }

    public int getEinkommen() {
        return einkommen;
    }

    // Regel 1: gesamtes Jahreseinkommen wird versteuert
    public int zuVersteuerndesEinkommen() {
        return einkommen;
    }

    // Regel 4 ueberschreibt nur diesen Wert (Adel: 20)
    protected int mindestSteuer() {
        return 1;
    }

    // Regel 2: 10 %, abgerundet, mindestens mindestSteuer()
    public int steuer() {
        int steuer = zuVersteuerndesEinkommen() * STEUERSATZ_PROZENT / 100;
        return Math.max(steuer, mindestSteuer());
    }
}
```

```
package aufgabe3;

public class Koenig extends Einwohner {
    public Koenig(int einkommen) { super(einkommen); }

    @Override
    public int steuer() { return 0; }              // Regel 3
}

public class Adel extends Einwohner {
    public Adel(int einkommen) { super(einkommen); }

    @Override
    protected int mindestSteuer() { return 20; }   // Regel 4
}

public class Bauer extends Einwohner {             // Regel 6: keine Abweichung
    public Bauer(int einkommen) { super(einkommen); }
}

public class Leibeigener extends Bauer {
    public Leibeigener(int einkommen) { super(einkommen); }

    @Override
    public int zuVersteuerndesEinkommen() {        // Regel 5: 12 Taler steuerfrei
        return Math.max(getEinkommen() - 12, 0);
    }
}
```

b)

```
package aufgabe3;

public class Finanzamt {
    public static int berechneSteuerEinnahmen(Einwohner[] einwohner) {
        int summe = 0;
        for (Einwohner e : einwohner) {
            summe += e.steuer();
        }
        return summe;
    }
}
```

c)

```
package aufgabe3;

public class Main {
    public static void main(String[] args) {
        Einwohner[] einwohner = {
            new Koenig(1_000_000),
            new Adel(30_000),
            new Adel(150),
            new Bauer(2_000),
            new Leibeigener(150),
            new Leibeigener(5)
        };
        int summe = Finanzamt.berechneSteuerEinnahmen(einwohner);
        System.out.println("Gesamte Steuereinnahmen: " + summe + " Taler");
    }
}
```

Einwohner	Einkommen (Taler)	Steuer (Taler)	Berechnung
Kaiser	1.000.000	0	Regel 3: keine Steuer
Adliger (reich)	30.000	3.000	10 % von 30.000
Adliger (arm)	150	20	10 % = 15, Mindeststeuer Adel 20
Bauer	2.000	200	10 % von 2.000
Leibeigener 1	150	13	(150 - 12) = 138, 10 % = 13,8 abgerundet 13
Leibeigener 2	5	1	zu versteuern 0, Mindeststeuer 1

Gesamte Steuereinnahmen: 0 + 3.000 + 20 + 200 + 13 + 1 = **3.234 Taler**

Aufgabe 4

4 Lösung: Abstrakte Klassen & Interfaces (Musterlösung)

1. *Was ist der Hauptunterschied zwischen einer abstrakten Klasse und einem Interface?*
   Eine abstrakte Klasse kann Zustand (Attribute), Konstruktoren und bereits implementierte Methoden enthalten und stellt eine gemeinsame Basis für verwandte Klassen dar („ist-ein“). Ein Interface beschreibt nur einen Vertrag (welche Methoden eine Klasse anbieten muss); es hat keinen Objektzustand (nur Konstanten) und bis Java 7 nur abstrakte Methoden (seit Java 8 zusätzlich default- und static-Methoden).
2. *Können abstrakte Klassen Mehrfachvererbung unterstützen?*
   Nein. Java kennt keine Mehrfachvererbung bei Klassen, auch nicht bei abstrakten: Eine Klasse kann nur **eine** Oberklasse (`extends`) haben. Mehrfachvererbung des Typs ist nur über Interfaces möglich (`implements A, B`).
3. *Wie können abstrakte Methoden erstellt und implementiert werden?*
   Eine abstrakte Methode wird mit dem Schlüsselwort `abstract` und ohne Rumpf deklariert: `abstract void hupen();` (nur in abstrakten Klassen). Implementiert wird sie in der ersten konkreten Unterklasse durch Überschreiben (`@Override`); sonst muss auch die Unterklasse abstrakt sein. Interface-Methoden sind automatisch abstrakt und werden mit `implements` umgesetzt (dort `public`).
4. *Wann ist es sinnvoll, eine abstrakte Klasse anstelle eines Interfaces zu verwenden?*
   Wenn die Klassen gemeinsamen Zustand oder gemeinsamen Code teilen sollen (Attribute, Konstruktoren, konkrete Methoden), eine enge „ist-ein“-Beziehung besteht, `protected`-Zugriff oder eine Vorlagenmethode (Template Method) gebraucht wird. Ein Interface passt besser, wenn nur eine Fähigkeit festgelegt wird, die auch unverwandte Klassen haben können.
5. *Kann eine Klasse gleichzeitig eine abstrakte Klasse erweitern und ein Interface implementieren?*
   Ja. Die Klasse erweitert genau eine (abstrakte) Klasse und implementiert beliebig viele Interfaces: `class Auto extends Fahrzeug implements Fahrbar`.
6. *Können Variablen in einem Interface deklariert weren?*
   Ja, aber nur Konstanten: Variablen in Interfaces sind automatisch `public static final` und müssen sofort initialisiert werden. Instanzvariablen (Zustand) gibt es in Interfaces nicht.
7. *Gibt es Unterschiede in der Sichtbarkeit (Zugriffsmodifizierer) von Methoden und Feldern in abstrakten Klassen und Interfaces?*
   Ja. In abstrakten Klassen sind alle Modifizierer möglich (`public`, `protected`, package-private, `private`) für Felder und Methoden; abstrakte Methoden dürfen aber nicht `private` sein. In Interfaces sind Methoden implizit `public` (ab Java 9 auch `private` Hilfsmethoden), Felder immer `public static final`.

Kurzes Beispiel (Package aufgabe4)

```java
package aufgabe4;

interface Fahrbar { void fahren(); }

abstract class Fahrzeug {
    protected int geschwindigkeit;               // Zustand ist erlaubt
    abstract void hupen();                       // abstrakt: kein Rumpf
    void stoppen() { geschwindigkeit = 0; }      // konkret: gemeinsamer Code
}

class Auto extends Fahrzeug implements Fahrbar { // 1x extends, n x implements
    @Override void hupen() { System.out.println("Hupe!"); }
    @Override public void fahren() { geschwindigkeit = 50; }
}
```